PROJECT PROPOSAL

Travel Style Segmentation & Drift Monitoring

ITCS355 - Machine Learning Operation and Deployment | Cloud Platform: Microsoft Azure
Perawis Buranasing (6688012) | Supitsara Tanasansukstid (6688121) | Atichat Kangsamut (6688193)

1. Problem and Users

Travelers may prefer very different activities, but these preferences are difficult to summarize consistently. We will build a low-stakes service that groups a complete 24-category rating profile into an interpretable travel segment. The primary user is a travel platform that needs an immediate first-step segment during an interactive personalization flow. Travelers and tourism analysts are secondary users. Segments describe preference patterns only; they are not personality labels and will not support high-impact decisions.

2. Dataset and Data Quality

We will use UCI's Travel Review Ratings dataset: 5,456 profiles with average ratings for 24 attraction categories in Europe, licensed under CC BY 4.0 (DOI: 10.24432/C5C31Q). User ID will support repeatable splitting only. Data and code will be versioned with DVC and Git. Our initial audit found a trailing empty column and two malformed records, which will be quarantined deterministically. Because zeros appear although the metadata describes ratings as 1-5, we will compare and document zero-value policies instead of guessing their meaning.

3. Machine Learning Approach

K-Means will be the main model because there are no fixed travel-style labels. A Gaussian Mixture model will provide a simple comparison. We will select the number of clusters using stability across seeds, Silhouette score, cluster size, and interpretability rather than one score alone. MLflow connected to Azure Machine Learning will track each experiment. The model, scaler, feature order, centers, segment-name mapping, Git commit, data version, and run ID will be registered together so every prediction can be traced to its source.

4. Serving and Requirements

We will deploy a Dockerized FastAPI service on Azure Container Apps because the travel platform must return a segment during the same personalization interaction. The API will support one profile, batches of up to 100, and separate health and readiness checks. Our pre-registered target is warm p95 latency below 300 ms at concurrency 10; scale-to-zero cold starts will be measured and reported separately. Profiles far from their assigned centroids will be returned as uncertain rather than receiving a confident segment. Each prediction will use the complete 24-category profile supplied in the current request, and the service will not reuse a cached segment after the profile changes. Model freshness is handled separately: retraining will require an approved data version or persistent validated drift.

5. Monitoring and Planned Failure

Azure Monitor and Application Insights will show request and error rates, p50/p95/p99 latency, cold starts, model version, segment distribution, distance-to-centroid statistics, invalid and uncertain requests, and feature drift. The frozen training split will be the reference window. Recent traffic or deterministic replay will be the comparison window, with a minimum sample size before alerts can fire.

Our deliberate failure is an incorrect cluster-to-segment-name mapping after retraining. Cluster numbers can change even when patterns remain similar. We will deploy a bad mapping as a clearly tagged Azure Container Apps staging revision and replay known profiles. A semantic mapping check and output-distribution monitor should alert us. We will inspect the model version and logs, route traffic back to the last-good revision, verify recovery, and add a promotion test that aligns new centers with existing names or blocks ambiguous mappings.

6. CI/CD and Cost

GitHub Actions will run linting, unit tests, data-contract tests, model-behavior and mapping tests, a container build, and an API integration test. A green main branch will authenticate to Azure through OIDC, push an immutable Git-SHA image to Azure Container Registry, and deploy only to staging. Smoke and load tests must pass before a manually approved promotion. Long-lived cloud credentials will not be stored in GitHub or the image.

The Azure path is Azure Blob Storage/DVC to Azure ML/MLflow, then Azure Container Registry, Container Apps, and Azure Monitor. The CPU-only service will use the Consumption plan with minimum replicas set to zero. Pre-run assumptions are at most 60 CPU-minutes for training/tuning, no more than 10,000 demo requests, and below 2 GB of stored data, images, and logs. Before student credit, we reserve 70 THB for compute and serving, 25 THB for storage, registry, and monitoring, and 55 THB contingency: a 150 THB ceiling. We will set a budget alert, export actual cost, calculate cost per 1,000 predictions, and remove test resources after preserving required evidence.

7. Team Responsibilities

Perawis Buranasing: data, DVC, preprocessing, training pipeline, Azure ML/MLflow, and model registry.

Supitsara Tanasansukstid: FastAPI, Docker, Azure Container Apps deployment, CI/CD, and load testing.

Atichat Kangsamut: dashboard, drift detection, alerting, failure demonstration, cost report, and model card.

All members will review segment names, tests, documentation, the final repository, and the live demonstration.

Dataset citation: Renjith, S. (2018). Travel Review Ratings. UCI Machine Learning Repository. https://doi.org/10.24432/C5C31Q